

## 1.3 Code Wizards

Chat-based AI tools provide an uncomplicated and free introduction to the world of AI-supported coding. However, the constant back and forth between the web interface of the AI tool and the editor or development environment is somewhat inconvenient. You constantly need to copy code back and forth between the editor and the web browser, and surely that must be possible in an easier way.

In fact, Microsoft had already started the technical preview of *GitHub Copilot* before OpenAI presented its universal ChatGPT interface to the public. GitHub Copilot has been available as a paid service since June 2022. The cheapest option currently costs \$10 per month.

---

### The “Copilot” Concept

Microsoft initially only used Copilot to describe the new code wizard. However, the term is now used widely for various Microsoft AI tools, for search functions, as well as for Microsoft Office tools. Since mid-2024, there has even been a notebook series with this name (Copilot+ PC). This book focuses exclusively on the GitHub Copilot.

The idea of GitHub Copilot is a logical extension of the IntelliSense concept that has long been established in the Microsoft world. While entering code, Visual Studio Code (VS Code) suggests a completion for the entry. But while IntelliSense was limited to the names of methods or properties, GitHub Copilot makes suggestions for loops, including their content, or even suggestions for entire methods or functions.

GitHub Copilot sometimes works incredibly well. Say you start the definition of a new function such as `findCancelledCustomers`. From the context of the rest of the file, GitHub Copilot recognizes that you probably want to get a list of all customers from the `customers` table whose status is **Canceled**.

Accordingly, GitHub Copilot proposes the complete code for the function: it generates SQL code for the query, executes using the variable already defined elsewhere in the file for the database connection, and returns the list of customer names and IDs as the function result.

Of course, GitHub Copilot's suggestions aren't always perfect. You can often use the code anyway and then have to adjust a few details. If the AI tool doesn't correctly guess your intention, simply continue entering your code manually. In the subsequent line or the line after that, GitHub Copilot will then make new suggestions that are better due to additional context information (e.g., as comments or variable names).

Behind the scenes, GitHub Copilot, like ChatGPT, currently uses a language model from OpenAI (most recently GPT-4). The language model was trained using code from GitHub's public repositories, among other things. This fact has led to controversies concerning both copyright issues and security aspects. It's still not entirely clear whether training with code from public sources is actually considered *fair use*, as argued by GitHub and Microsoft. In this book, we address this question and other aspects in [Chapter 13](#).

---

## Alternatives to GitHub Copilot

Just as ChatGPT isn't the only chat-based AI environment, there are also various alternatives to GitHub Copilot, such as GitLab Code Suggestions with *Duo Chat* or the *Swift Assist* function integrated in current Xcode versions.

*Cody* is extremely exciting. Basically, Cody also completes code and answers chat prompts. But it has two main advantages: First, you can choose between different language models (local large language models [LLMs], Claude Sonnet, Gemini, Mixtral, and GPT-4o and Cloud Opus with the Pro account). Second, Cody takes into account your entire code base in the active directory. (This function is expected to also be implemented in GitHub Copilot sooner or later.) You can try Cody free of charge. The Pro subscription with unlimited chat functions and a larger selection of LLMs currently costs \$9 per year. You can find more information on this here:

<https://sourcegraph.com/cody>.

Apart from commercial offers, there are also free editor extensions and plug-ins. They require an AI system to be accessible via an interface (API). In this book, we'll focus on the VS Code plug-in *Continue* as an example (see [Chapter 9, Section 9.5](#)). It provides very similar functions to GitHub Copilot, but can use a locally executed language model. Provided you have the appropriate computing power, you can avoid having to pay for AI providers and avoid transferring your data to the cloud.

Continue can also be connected to an AI instance in the local network or on the internet. For example, you can configure a powerful computer with a GPU as a workgroup; all team members then use this computer together for AI coding tasks. A major advantage for companies is that they can avoid any data protection concerns.

### 1.3.1 Installing GitHub Copilot

GitHub Copilot was initially developed as an extension to the VS Code editor. The combination of VS Code with GitHub Copilot is still extremely popular today. Accordingly, most of the code wizard examples in this book are applications of this “dream team.” Before you can use GitHub Copilot, you need to install the extension of the same name, log in to your GitHub account in VS Code, and sign up for a Copilot subscription on the GitHub website (<https://github.com/features/copilot/plans>).

You can try GitHub Copilot free of charge for 30 days. Free licenses are provided for students, teachers, and maintainers of large open-source projects.

---

### Editors Compatible with GitHub Copilot and Development Environments

GitHub Copilot can be used not only with VS Code but also with various other editors or development environments, such as Visual Studio, all JetBrains IDEs (IntelliJ, Android Studio, PyCharm, etc.) as well as the Vim and Neovim editors. There are unofficial plug-ins for various other editors, such as Emacs or Xcode.

## 1.3.2 First Steps with Visual Studio Code and GitHub Copilot

After the installation, VS Code displays a Copilot icon in the status bar that leads you to the Copilot status menu. The first line of the menu must contain the **Status: Ready** entry.

To try out GitHub Copilot, create a new file in your favorite language. We've used Python for the following example. Now enter `def factorize(x)`, for example. It can be assumed that you want to program a function that breaks down an integer into its factors.

GitHub Copilot suggests a possible continuation of the code in gray font. If you want to accept the entire proposal, simply confirm it by pressing the `[Tab]` key. Alternatively, you can repeatedly press `[Ctrl] + [▶]` to accept the suggestion in small parts (word by word or line by line). The mini dialog for proposal selection only displays when you move the cursor over the code proposal.

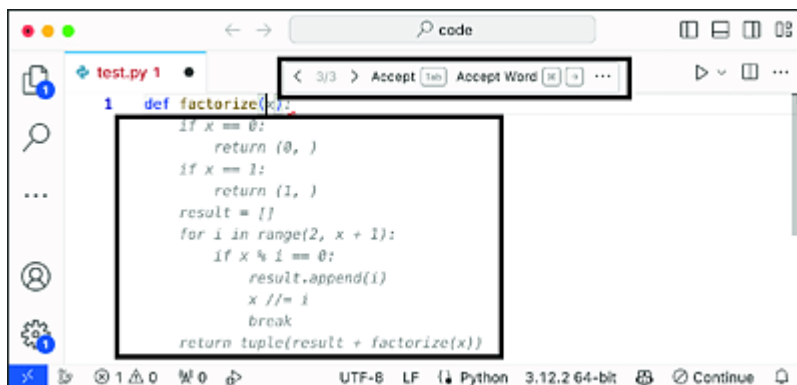


Figure 1.3 First Experiments with GitHub Copilot in VS Code

## 1.3.3 Switching between Code Suggestions

VS Code always shows only one code suggestion, although GitHub Copilot internally offers several options to choose from. The shortcut keys `[Alt] + [↓]` or `[Alt] + [↑]` are used to switch between the first three suggestions.

### Keyboard Shortcut

### Function

`[Tab]`

Accepting a suggestion

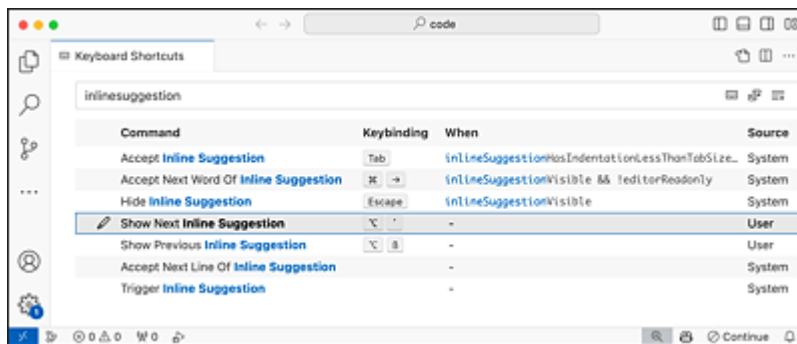
| Keyboard Shortcut                                                               | Function                                   |
|---------------------------------------------------------------------------------|--------------------------------------------|
| <code>Ctrl</code> + <code>▶</code> or<br><code>cmd</code> + <code>▶</code>      | Accepting a word or line of the suggestion |
| <code>Alt</code> + <code>[</code>                                               | Next suggestion (US keyboard)              |
| <code>Alt</code> + <code>]</code>                                               | Previous suggestion (US keyboard)          |
| <code>Ctrl</code> + <code>Enter</code> or <code>cmd</code> + <code>Enter</code> | Displaying 10 suggestions in a side pane   |
| <code>Ctrl</code> + <code>I</code> or <code>cmd</code> + <code>I</code>         | Opening the Copilot dialog (inline chat)   |
| <code>Ctrl</code> + <code>Alt</code> + <code>I</code>                           | Opening the chat sidebar                   |
| <code>Ctrl</code> + <code>cmd</code> + <code>I</code>                           | Opening the chat sidebar (macOS)           |

**Table 1.2** VS Code Shortcuts for Controlling GitHub Copilot

These shortcuts aren't defined on Linux and macOS, but you can change that by means of a manual configuration.

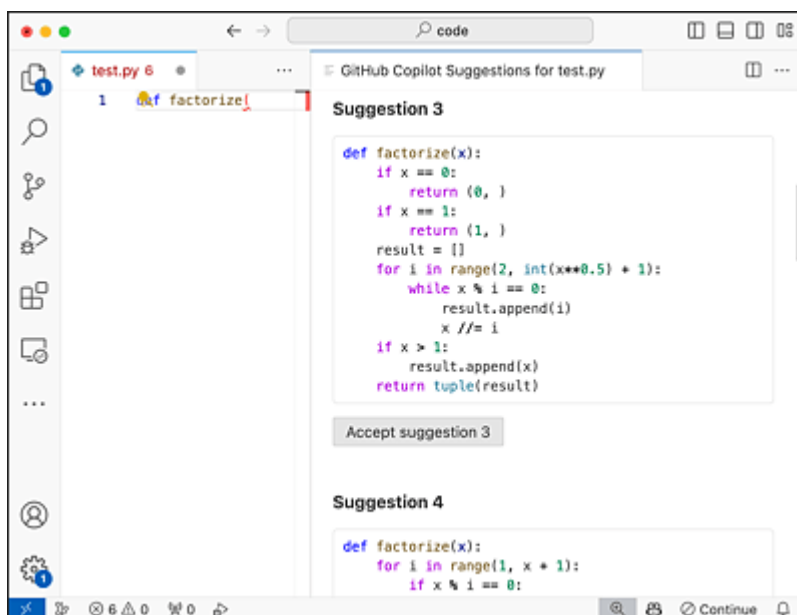
We opened the **Keyboard Shortcuts** configuration dialog in VS Code, searched for the `editor.action.inlineSuggest.showPrevious` and `.showNext`

commands, and assigned our own keyboard shortcuts to them by double-clicking in the **Keybinding** column.



**Figure 1.4** Defining Custom Shortcuts for Selecting GitHub Copilot Suggestions

If three suggestions aren't enough, you can press `Ctrl` + `Enter` to open a pane on the right-hand side with up to 10 suggestions. You can let yourself be inspired by the ideas and then select a suggestion by clicking on the **Accept suggestion n** button.



**Figure 1.5** Code Suggestions in the Pane on the Right

### 1.3.4 Tips and Tricks

When testing in a new, almost empty code file, a significant part of the potential of GitHub Copilot remains unused. We therefore recommend that you familiarize yourself with the AI tool in a project whose code you know well. It's crucial that you can assess the quality of GitHub Copilot's suggestions beyond doubt. This works best in a familiar environment. Just add

a new feature to your project or make changes to existing code, and you'll be amazed at how constructive and useful the suggestions are!

To exploit the full potential of GitHub Copilot, consider the following tips:

- Give new functions or methods names that are as meaningful as possible.
- If necessary, formulate a comment before the function definition, before a loop, or before a branch (one or two lines are sufficient) to describe the task more precisely or to point out special features or details. This will help GitHub Copilot make more appropriate suggestions in context.
- You can also take the opposite approach by writing code first (with or without AI support) and then letting GitHub Copilot help you write the comment. Simply start the comment with #, //, or /\*, depending on the language. GitHub Copilot parses the surrounding code and suggests documentation that is often useful.
- Open a few relevant other files of your project in VS Code with comparable code or with the definition of functions or classes that you want to use in the file you're currently working on. GitHub Copilot considers the code of the open tabs as context information and recognizes how you've called or evaluated functions, properties, or methods so far. This improves the quality of the code proposals.

Currently (early 2025), GitHub Copilot only considers project files that are open in the editor. It's possible that future versions will also analyze closed files. This is currently not working for two reasons: First, the size of the context is very limited with the current language models and GitHub Copilot simply can't take all files into account. Secondly, with large projects, it's difficult to identify the relevant files within hundreds of files.

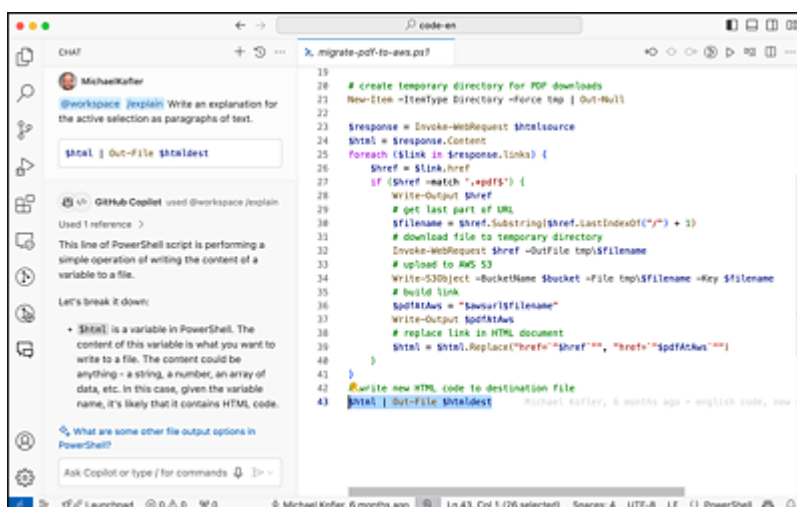
### 1.3.5 Chat Mode with Additional Keywords

The code wizard and the chat functions are by no means mutually exclusive. Press `Ctrl + Alt + I` or `ctrl + cmd + I` to open the chat sidebar within the editor. There you can communicate freely with the AI language model. You can transfer the code contained in the response directly to the

currently open code file by clicking a button at the current cursor position, thus saving yourself the detour via the clipboard.

A decisive advantage compared to ChatGPT in the web browser is that you can refer to the context of the open editor tabs when communicating via special keywords, as follows:

- @workspace indicates that you're referring to the code in the workspace and don't want to ask a general question. (The VS Code workspace is made up of the files in the project directory or all currently active projects.)
- @vscode means that you're asking a question about using the editor. GitHub Copilot may include VS Code commands in the response that you can click on directly.
- @terminal directs the context to the terminal, which is useful for questions relating to terminal commands. Depending on the operating system used, GitHub Copilot explains answers to Linux and macOS commands or PowerShell cmdlets.
- /clear deletes the preceding chat history.
- /doc adds comments to the code.
- /explain explains the task or function of the current code. This is particularly useful if you're editing third-party code or have taken code from the internet (e.g., from Stack Overflow) and don't fully understand it.
- /fix tries to fix problems or errors in the selected code.
- /tests generates unit tests for the current function or method.



**Figure 1.6** GitHub Copilot Explaining the Highlighted Line of Code in the Chat Sidebar



---

## Tip

The `/explain`, `/fix`, `/doc`, and `/tests` commands can be run even more conveniently directly in the code window. To do this, select the relevant lines in the code, and execute one of the following context menus: **Copilot • Explain This, Fix This, Generate Docs, or Generate Tests**.

Documentation and unit tests are covered in detail in [Chapter 6](#) and [Chapter 5](#), respectively.

### 1.3.6 Chat Examples

The following prompts show some examples of how the keywords are used. In the first example, VS Code is supposed to explain the code just highlighted.

**Prompt:** @workspace /explain the selected lines.

The selected lines are from a shell script and perform the following operations . . .

You can use GitHub Copilot to search for specific files in the project. Our tests produced mixed results. The CSS search was successful, while the search for unsafe full-text passwords failed.

**Prompt:** @workspace Which CSS file defines the `invoiceDate` class?

The `invoiceDate` class is defined in the `css/invoice.css` file.

**Prompt:** @workspace Which scripts contain unsafe clear text passwords for database connections?

[no useful results]

If you're lost in the plethora of VS Code commands, GitHub Copilot can sometimes help you out. Again, here's a positive and a negative example. The answer to the second prompt is incomplete because GitHub Copilot doesn't provide the option of activating the secondary panel and simply

dragging the chat there (see also the corresponding “Making the Best Use of Space” box).

**Prompt:** @vscode How can I see the differences in the code file since the last commit?

You can use the **Git: Open Changes** (`git.timeline.openDiff`) command to show the differences in the active document compared to the last commit.

**Prompt:** @vscode I want to display the chat panel on the right side. All other panels should remain at the left side.

You can use the **View: Move Panel Right** command to move the panel to the right. Then, use the **View: Show Chat** command to open the chat panel.

Please note that this will move all panels, including the chat panel, to the right side. VS Code currently doesn't support displaying panels on different sides simultaneously.

When executing the following question, we've just edited files on a Linux server in VS Code. We particularly liked the correct one-line answer without any superfluous blah-blah.

**Prompt:** @terminal How can I find files that haven't changed for more

```
find . -type f -mtime +180
```

You can't directly change chat commands once they have been executed. Instead, you need to scroll through previously executed prompts using the cursor keys as in the terminal, change them, and then execute them again.

---

## Making the Best Use of Space

If the chat in the VS Code sidebar is too small for you, you can open it in a separate window via **Open Chat in Editor** or **Open Chat in Window**.

Another variant is to use **View: Toggle Secondary Side Bar Visibility** to show a second sidebar on the right-hand side and move the chat history there. This is particularly useful when you work on very wide monitors.

### 1.3.7 Alternatives to the Chat Sidebar

Instead of using GitHub Copilot in the chat sidebar, you can open a small inline chat dialog via `Ctrl + I` or `cmd + I`, respectively. You can formulate your question there and then transfer the answer directly into the code. The inline chat can be used very efficiently for small changes. In addition, the chat history remains organized.

You can also ask minor questions directly in the code. To do this, start a comment using `q:`, and then formulate your question. As soon as you press `Enter`, GitHub Copilot inserts another comment line that starts with `a:` and contains the answer:

```
// q: How do I call bash script from PHP?    <- Your  
input  
// a: Use shell_exec() or exec() function.    <- GitHub  
Copilot
```

This type of question/answer game is particularly suitable for small details. The biggest advantage is that you never have to leave the editor. As soon as

the problem has been solved, you can delete the comment lines containing the question and answer.

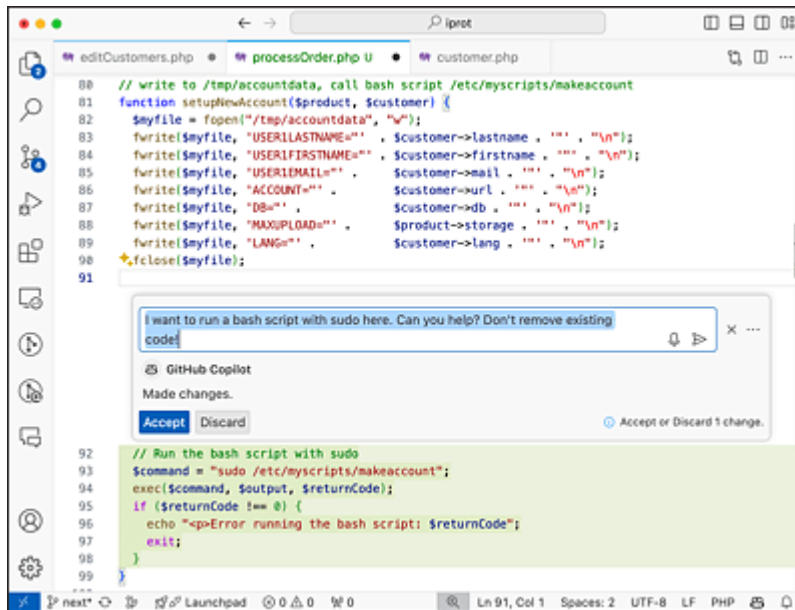


Figure 1.7 The Inline Chat Dialog

### 1.3.8 Other Functions

Microsoft is endeavoring to integrate GitHub Copilot as an all-encompassing tool in VS Code. You must decide for yourself to what extent these functions are useful in your workflow:

- **Git commits**

If you use Git version management, you can have the commit text written by GitHub Copilot. The AI tool attempts to summarize the last changes made. Sometimes, this works quite well, and, at other times, you just need to change the text. To us, the messages tended to be too lengthy.

- **Process error message**

If an error occurs during the execution of a program, VS Code displays a **Copilot** icon in the terminal area next to the error message. If you click on it, GitHub Copilot will try to explain the cause of the error in the chat. Ideally, it also provides a solution straight away and executes the suggestion with a further click.

Our experience with this feature was mixed. For some programming languages (including Python!), GitHub Copilot doesn't even recognize completely trivial errors such as missing parentheses. Admittedly, there

are also cases in which the function works as advertised. But there is certainly still room for improvement here.

- **GitHub Copilot in the terminal**

Once you've executed a command in the terminal, you can use GitHub Copilot to have this command explained to you.

If an error occurred during the execution of the last command, GitHub Copilot explains the cause of the error and suggests a corrected or improved command.

We've already pointed out the possibility of asking questions in the chat with `@terminal` that relate specifically to terminal commands.

### 1.3.9 Security and Data Protection

You should be aware that GitHub Copilot has access to all files in your project. This is necessary for the tool to work. The access also includes files with passwords, license keys, and so on. You must therefore take a great leap of faith with GitHub Copilot that it won't misuse this data.

Ultimately, however, this consideration applies not only to AI tools but also to every editor and every development environment: a backdoor in the IDE or a plug-in with malicious code can also leak company secrets. You should therefore think carefully about which plug-ins you install in VS Code or the IDE of your choice. It's actually surprising that there has never been a (known) safety disaster in this regard.

---

#### Never Save Passwords in Code Files!

Irrespective of the preceding considerations, you should never save passwords directly in code files. Use separate configuration files such as `config.json` or `.env`, or save passwords in environment variables that your code can access.

Of course, VS Code also has access to password files. In most cases, however, their evaluation by GitHub Copilot or other AI tools isn't necessary. However, if a password is in plain text in the code file, it will inevitably be uploaded to external servers when AI tools are used. Even if the AI providers themselves have no interest in these passwords, their uncon-

trolled storage in any database to improve or train future language models is a security nightmare.

### 1.3.10 Trouble in Remote Mode

The *Remote SSH* plug-in allows you to use VS Code to also edit code in directories located on other computers, provided you have Secure Shell (SSH) access on the external computers. In our tests, however, GitHub Copilot refused to cooperate in such cases. The following lines in the `settings.json` configuration file on your local computer can help:

```
# include in settings.json
...
"remote.extensionKind": {
  "GitHub.copilot": ["ui"],
},
...
```

The easiest way to open the `settings.json` file is by using **Open User Settings (JSON)** (`F1`). You can find more details on that process here: <https://github.com/orgs/community/discussions/6942#discussioncomment-4962858>.